IBM Z Datathon 2026 Team Guide

OutOfRole insider threat detection

The Debugging Table

Prepared October 16 2026 | Team edition

Use this guide to prepare one LinuxONE server for our team, run the IBM reference labs, build OutOfRole during the event, and assemble a reproducible final submission. The project learns normal user and resource relationships, ranks unusual activity, and uses a local IBM Granite model to explain evidence for a human analyst.

The primary source is the LinuxONE Setup and Tech Prep Session presented by Alex Osadchyy and Bruce Gilkes for the Shooting Stars Foundation. Its technical details are preserved in the topic transcript in Appendix A. The working instructions cross-check that recording against IBM-linked Community Cloud guides and the official event website.

Watch the primary workshop recording

How to read the evidence labels

Label	Meaning
REQUIRED	An explicit requirement in the official event page or FAQ. A video-only instruction is labeled REQUIRED IN SESSION so its authority remains visible.
PRESENTER	A recommendation, demonstration, allocation, or claim from the recording or its slides. It is not automatically an eligibility rule.
TEAM SUGGESTION	An added operational or OutOfRole implementation choice. These choices can be changed by the team.
VERIFY	Unresolved, conflicting, inaccessible, or not tested on our actual VM. Check the stated source or organizer before relying on it.

Starting material: Data_Access_Guide.md and Abstract_Submission.md in this workspace. This edition carries forward their data sources, download details, project concept, audience, and ALS approach. A separate file titled Team Guide was not present. The original files have been preserved.

Verification limit: commands were checked against documentation and inspected for syntax; no LinuxONE account was created and no VM deployment or model benchmark was executed for this document. The recovered audio and video each run 55 minutes 6 seconds, while YouTube metadata reports 55 minutes 28 seconds. Appendix A uses recovered-media time; the 22-second discrepancy is unresolved.

Contents

1 Event requirements and deadlines....3

2 Team roles and minimum result....4

3 Register and create the LinuxONE VM....5

4 Configure SSH and prove the platform....6

5 Install Docker and start Jupyter Lab....7

6 Run the reference labs and local Granite....9

7 Obtain and inspect the CERT data....10

8 Configure PySpark for OutOfRole....13

9 Implement OutOfRole during the event....14

10 Troubleshooting and recovery....18

11 Judging evidence and final submission....20

Appendix A Cleaned topic transcript with timestamps....22

Appendix B Verification and unresolved items....28

Appendix C Source links and evidence register....29

Start with Sections 1 and 2. The infrastructure owner follows Sections 3 through 6. Data and model owners follow Sections 7 through 9. Everyone reviews Sections 10 and 11 before recording the demo. Appendix A preserves the workshop; Appendix B records source conflicts and verification; Appendix C lists the source links.

1 Event requirements and deadlines

REQUIRED [S2]: each participant registers individually; the team lead completes team registration. The public timeline explicitly requires 4 to 6 team members, although one FAQ calls that size recommended. Use 4 to 6 and keep the team roster consistent across the portal and submission.

1. Confirm that every member is a currently enrolled college or university student. The organizer hub states age 18 or older and participation subject to its official rules. Do not infer eligibility from hidden platform age metadata.
2. Confirm the team registration, member list, selected track, and accepted abstract in Hack2skill. Our abstract concept is OutOfRole under AI Secured Innovation Without Exposure.
3. Complete the October 16 LinuxONE readiness setup checks. The event FAQ requires attendance at the session and warns that later technical problems may not qualify for extensions. If you missed the live session, review its recording for setup and ask the organizer whether that meets the attendance requirement; equivalence was not verified.
4. Join the official event Discord through the invitation provided in the participant dashboard or organizer communication. Use its IBM SME helpdesk for technical support. No verified public invitation URL was recovered, so this guide does not invent one.
5. Prepare the problem, datasets, abstract, architecture, tools, and environment before the event. Build the actual solution code during the official event window. The FAQ disallows continuation projects and reuse of a prior project as the core submission; it allows properly licensed open-source libraries, public APIs, and IBM tools.
6. Demonstrate why the solution runs on IBM Z or LinuxONE. Capture actual execution evidence and submit every field required by the live final-project form before its closing time.

Published schedule and conflicts

All conversions below use UTC, India Standard Time at UTC plus 5 hours 30 minutes, and America Chicago daylight time at UTC minus 5 hours on these dates. The source is the official event page public record retrieved October 16 [S2]. Its event-specific Timeline entries are more specific than the top-level submissionEnd field, which currently matches abstract submission.

Milestone	UTC	Chicago CDT	Status
Readiness session start	Oct 16 13:45	Oct 16 08:45	Timeline; hub advertises 14:00 UTC instead
Registration and abstract close	Oct 16 18:29	Oct 16 13:29	Current timeline; conflicts with older FAQ dates
Team formation closes	Oct 15 18:29	Oct 15 13:29	Current timeline
Global kickoff starts	Oct 17 03:15	Oct 16 22:15	Timeline; hub advertises 03:30 UTC
Project submissions open	Oct 17 04:30	Oct 16 23:30	Specific project timeline entry
Virtual closing ceremony	Oct 18 05:00	Oct 18 00:00	Timeline; does not establish coding cutoff

Milestone	UTC	Chicago CDT	Status
Project submissions close	Oct 18 16:30	Oct 18 11:30	Timeline; 22:00 IST; verify live final form
Feedback survey closes	Oct 23 18:29	Oct 23 13:29	Timeline
Career panel and winners	Oct 28 time not stated	Time not stated	FAQ; confirm organizer notice

VERIFY: the FAQ still says registration October 1 and abstract October 5. Our local abstract file records October 15 at 13:29 CDT. These are historical or conflicting statements, not proof that our submission is accepted. The organizer hub advertises a 24-hour competition, while the current project-upload window spans 36 hours. Do not treat the upload extension as permission for extra coding hours. Confirm the coding cutoff and final form deadline in official announcements and retain a screenshot.

TEAM SUGGESTION: target a completed recording, tested code link, and draft submission by October 17 at 22:30 CDT, then submit as soon as the portal permits. This is our conservative internal target, not an event deadline.

2 Team roles and minimum result

TEAM SUGGESTION: keep the first result small enough to finish. Demonstrate unusual workstation use and removable-media behavior using CERT r4.2. Compare a simple baseline with implicit-feedback ALS, then generate an evidence-grounded local explanation. A ranked alert supports analyst review; it does not establish that an employee is malicious.

Owner	Responsibilities	Handoff
Team lead	Roster, announcements, coding window, final form, receipt	Confirmed deadlines and submission checklist
Infrastructure owner	One VM, SSH access, Docker, Jupyter, backups	Working connection and runtime evidence
Data owner	Checksums, schemas, time split, LDAP snapshots, answer key	Versioned feature data and evaluation labels
Detection owner	Baseline, ALS, cold starts, thresholds, evaluation	Ranked alerts and measured metrics
Explanation owner	Local Granite, prompts, evidence checks	Validated explanations with source event IDs
Demo owner	Analyst view, figures, recording, deck	Clear demonstration with every contribution credited

Combine roles if the team has four people. Choose one person to operate VM lifecycle controls so teammates do not stop, delete, or replace a machine during another person's work.

Minimum acceptance checklist

- A real LinuxONE VM returns s390x from uname -m, and the video shows our own workload executing there.
- The pipeline reads at least logon.csv, device.csv, file.csv, and dated LDAP metadata; it explains what each source actually measures.
- Training precedes validation and testing in time. Answer-key information is used only for evaluation.
- Both established users and new users or resources have a documented scoring path; cold starts are counted.
- An analyst can inspect an alert, its contributing events, peer context, and a cautious explanation.
- README, exact dependency versions, run order, data instructions, metrics, limitations, and final links are reviewable.

3 Register and create the LinuxONE VM

PRESENTER [V 07:47 to 26:42]: the event allocation is one server with 3 virtual CPUs, 8 GB RAM, 50 GB disk, and a 30-day trial. These are session statements; verify the assigned flavor in the dashboard. Bruce explicitly asks for one VM per team to conserve shared capacity. Treat that request as our operating rule.

1. Open the IBM LinuxONE Community Cloud landing page [S4], choose Try Virtual Machines, or use the registration link in Appendix C. LinuxONE Community Cloud is distinct from paid IBM Cloud VPC; this workflow uses the Marist-hosted portal.
2. Enter an email address you can access, create and remember a strong portal password, and complete your name, university, country, and required fields. IBM currently posts an Outlook and Hotmail registration issue [S4]; use a university or other working address if affected.
3. Describe OutOfRole in at least 75 characters, the minimum shown in the video. Example: Our student team will use synthetic CERT activity logs to detect unusual employee workstation and removable media behavior and explain alerts locally on LinuxONE.
4. Enter SSF2026 in the Event Code field. The live demonstration and current IBM lab README agree on that value. The final slide also says to put it in the project description; include it there too if useful, but do not omit the dedicated Event Code field when present.
5. Read the terms and complete the trial request yourself. Check the inbox and spam folder for the activation email. The presenter estimates seconds to 5 or 10 minutes during normal demand. Click the activation link; submitting the form alone leaves the account inactive.
6. Sign into the self-service dashboard using the credentials created at registration. Open Virtual Servers or Manage Instances. The View Guide link in the portal provides deployment help.
7. Create or import an SSH key pair. Import only your public key. If the portal generates a pair, securely save the downloaded private .pem file immediately; the deployment guide says it cannot be retrieved again. Keep the private key on your own laptop.
8. Choose Create, use an instance name such as outofrole2026 with no spaces or special characters, and select the event image. The live correction at 16:19 to 16:47 says to look for the Datathon suffix: Ubuntu 24.04 Datathon or RHEL 9.8 Datathon. The prerecorded RHEL 9.6 selection was superseded. Ubuntu is the path used in this guide.
9. Select the AI Lab flavor and the matching SSH key, review the allocation, then create the server. If AI Lab is missing, do not substitute the smaller Linux1 flavor. Contact linux1@us.ibm.com or the event SME; the presenter describes support deleting and correcting the registration so it can be repeated with SSF2026.

10. Wait for ACTIVE. The deployment guide lists NETWORKING and SPAWNING before ACTIVE; the presenter allows 15 to 20 minutes during mass provisioning. A newly created SHUTOFF instance may simply need Start. Record its current public IP, OS, resource allocation, expiry, and key name.

Expected result: exactly one team VM is ACTIVE with the event image and AI Lab resources. VERIFY: image availability, flavor names, trial expiry, and provisioning time can change. Do not delete a machine containing team work to correct an account issue without a backup and support guidance.

4 Configure SSH and prove the platform

REQUIRED BY CLOUD GUIDE [S5]: connect as linux1 using a matching SSH private key. Direct root login is disabled, and password authentication must not be enabled. The cloud grants sudo for administration and does not provide a server backup.

Mac or Linux laptop

1. Open Terminal on your laptop. Put the .pem file in your own .ssh folder. The following example assumes you saved it as outofrole2026.pem; substitute your actual filename.
2. Restrict permissions, then connect using the IP shown in the dashboard. In the sample command, replace VM_IP with that address. Do not type the quotation marks around an address or copy a presenter's demonstration IP.
3. On the first connection, verify the host fingerprint through the portal or an IBM SME before accepting it. If an old key warning appears after a rebuild, use the troubleshooting steps in Section 10.
4. Once connected, confirm the prompt is on the LinuxONE server. Run the platform and capacity checks below and save their output for the final demo.

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
chmod 600 ~/.ssh/outofrole2026.pem
ssh -i ~/.ssh/outofrole2026.pem linux1@VM_IP
```

Windows laptop

Use PowerShell with OpenSSH [S5]. Replace both the key path and VM_IP. If OpenSSH reports an unprotected private key, use the key file Properties Security Advanced dialog to disable inherited access and restrict access to your account; ask an SME if organizational policy blocks that change. Do not broaden key permissions to fix a login error.

```
ssh -i "C:\Users\YOUR_NAME\.ssh\outofrole2026.pem" linux1@VM_IP
```

PRESENTER: PuTTY is an alternative when PowerShell SSH is unavailable. Use PuTTYgen to import the .pem and save a .ppk, then set PuTTY host to the VM IP, login user to linux1, and select the private key under SSH authentication credentials. Follow the IBM-linked PuTTY guide [S6]. Do not rename a .pem to .ppk as a substitute for conversion.

Run these commands on the VM

```
whoami
uname -m
uname -a
lscpu
cat /proc/cpuinfo
free -h
df -h "$HOME"
lscpu | grep -i nnpa
```

Expected: whoami shows linux1; uname -m shows s390x. The video shows machine type 9175, z/VM virtualization, and NNPA available. That proves a platform capability in that demonstration, not acceleration of our ALS code. A missing nnpa match requires investigation; it is not proof that the SSH connection failed. Record actual results rather than copying the video's output.

Give teammates independent access

TEAM SUGGESTION: each teammate keeps a separate private key and supplies only a public key to the infrastructure owner. The owner can add authorized public keys to the shared linux1 account with appropriate permissions, following IBM support guidance. This retains the required public-key authentication. Do not distribute the owner's private .pem or portal password. If the team cannot configure this safely, have a designated operator run jobs while others collaborate through the repository.

For a verified rebuilt host only, remove that host's stale entry with ssh-keygen -R VM_IP and reconnect after checking the new fingerprint. This is a targeted replacement of Bruce's manual known_hosts cleanup, not an instruction to erase the whole known_hosts file or disable host-key checking.

5 Install Docker and start Jupyter Lab

PRESENTER [V 36:57 to 38:42; S7]: the IBM image bundles Jupyter and many AI frameworks. Use an s390x image, not an arbitrary amd64 image. The current lab repository specifies registry.linuxone.cloud.marist.edu/l1cc/jupyterlab-image-s390x:latest and the shared mount /home/ibm-user/shared. Old 2022 instructions use a different image path and /home/jovyan/shared; do not combine them.

VERIFY: the registry timed out from this authoring computer. The image URL is verified against current IBM lab documentation, but its pull, digest, size, and live behavior have not been tested on our VM. Reassign IMAGE in every new SSH session before using the image commands; shell variables are not saved by reconnecting.

Install on the Ubuntu VM

1. In the SSH session, confirm that Docker is not already installed with docker --version. If it is already supplied by the Datathon image, inspect its service before running another installer.
2. For a fresh Ubuntu VM, download the same Docker convenience script used in the IBM guide, inspect it, preview its actions, and run it. Docker describes this method as suitable for development environments [S10]. The commands below change the VM; run them on the VM, not the laptop.
3. Start Docker and add linux1 to the docker group. This group grants powerful access to the host. Exit the SSH session and reconnect so new group membership takes effect.
4. Run hello-world and verify Docker responds without a permission error. Then pull the IBM image and record its immutable digest for reproducibility.

```
curl -fsSL https://get.docker.com -o get-docker.sh
less get-docker.sh
sudo sh get-docker.sh --dry-run
sudo sh get-docker.sh
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
exit
# Reconnect with the SSH command from Section 4.
docker run --rm hello-world
IMAGE=registry.linuxone.cloud.marist.edu/l1cc/jupyterlab-image-s390x:latest
docker pull "$IMAGE"
docker image inspect "$IMAGE" --format '{{json .RepoDigests}}'
docker image inspect "$IMAGE" --format '{{.Architecture}}'
```

Expected architecture: s390x. Save the digest in the project environment notes; latest is a changing tag. If a pull asks for credentials or has no matching manifest, use the event helpdesk. The current README does not require the old workshop's shared registry login; no unverified password is supplied here.

Use a private Jupyter connection

TEAM SUGGESTION [S11, S12]: the video demonstrates public-IP HTTP on port 38888, opening a firewall port, and replacing the default token. For our security project, bind Jupyter to the VM loopback address and reach it through an SSH tunnel. This added setup protects the browser connection through SSH and avoids exposing Jupyter publicly. The shared directory persists notebook work outside the container.

1. Create ~/shared on the VM. Check the host user IDs with id and the image user IDs with the id command below. If the image user cannot write that folder, apply the targeted permission fix in Section 10. Avoid making the entire home folder world writable.
2. Generate a private token and start the container. The command overrides the image's notebook defaults with an explicit Jupyter Lab command, loopback address, port, and token. These settings follow Jupyter Server authentication documentation; verify the actual image accepts them.
3. Read the container logs and confirm it is listening on 127.0.0.1:38888. The token is a secret. Keep it out of screenshots, GitHub, and the submission video.
4. Open a second terminal on your laptop, create the SSH tunnel shown below, and keep it running. On Windows substitute the PowerShell key path from Section 4.
5. In your laptop browser open http://127.0.0.1:8888/lab. Enter the private token from the VM session. Create a notebook, write a short note in shared, save it, and confirm the corresponding file appears under ~/shared on the VM.

```
mkdir -p "$HOME/shared"
chmod 700 "$HOME/shared"
id
docker run --rm --entrypoint id "$IMAGE"
JUPYTER_TOKEN=$(openssl rand -hex 32)
docker run -d --name notebook --network host \
  -v "$HOME/shared:/home/ibm-user/shared" \
  "$IMAGE" jupyter lab --ip=127.0.0.1 --port=38888 \
  --no-browser --IdentityProvider.token="$JUPYTER_TOKEN"
docker ps -a
docker logs --tail 50 notebook
# Display privately when you need to enter the token in the browser.
printf '%s\n' "$JUPYTER_TOKEN"
```

Run this next command on the laptop, replacing VM_IP. -N creates a tunnel without a remote shell; -L maps laptop port 8888 to the VM's loopback port 38888.

```
ssh -N -o ExitOnForwardFailure=yes \
  -L 8888:127.0.0.1:38888 \
  -i ~/.ssh/outofrole2026.pem linux1@VM_IP
```

If 8888 is occupied, use 8889:127.0.0.1:38888 and open http://127.0.0.1:8889/lab. If you choose the presenter's public route instead, the IBM README opens TCP 38888 with iptables and uses http://VM_IP:38888. That route uses unencrypted browser HTTP; a unique token alone does not encrypt the connection. Do not combine public binding with the published default token Your_Token1. Do not add allow_origin=* or seccomp=unconfined unless a diagnosed issue and mentor guidance justify them.

Red Hat alternative

The current IBM RHEL guide [S8] uses Podman, sudo dnf -y install podman, the same image, and an SELinux :z suffix on the shared bind mount. Follow that branch consistently if the team selects RHEL.

Podman normally runs without a daemon; consult the upstream guide [S17] if the IBM example's podman.service start fails. The full command path in this document uses Ubuntu with Docker.

6 Run the reference labs and local Granite

PRESENTER [V 28:43 to 49:51]: run the supplied notebooks for orientation and adapt ideas to an original solution. The presenter's loan example combines predictive models, unstructured documents, policy retrieval, and a local Granite model. Copying an unchanged reference notebook is not an original Datathon project.

Notebook	What the video teaches	Use for OutOfRole
Fraud_LSTM_Keras_TF.ipynb	Historical sequences of seven transactions, two-layer LSTM, fraud scoring, portable model export	Think in temporal behavior; do not copy payment labels or metrics
Loan_Approval_Demo.ipynb	Structured risk score plus document features, policy retrieval, Granite reasoning and questions	Use a detector score plus retrieved evidence and local explanations
Churn_Scikit_PyTorch.ipynb	Data exploration, correlations, random forest and PyTorch logistic regression, PMML export	Use baselines, feature inspection, and clear explanations of the workflow
Digit_Class_TensorFlow.ipynb	Handwritten digit training, TensorFlow or Keras, ONNX export, NNPA benchmark	Learn runtime checks and benchmark reporting; images are not our input

1. Open one notebook from the Jupyter file browser, read its introduction, and run cells from top to bottom using Shift Enter. Wait until the running indicator finishes before launching another expensive cell.
2. Start with a small demonstration to confirm imports and file access. Use the notebook's own data and folders. Missing loan_demo modules or sample_data are a packaging or working-directory problem, not a reason to invent replacement model code.
3. In Loan_Approval_Demo.ipynb, inspect the Environment Setup cell. It imports load_model from loan_demo.rag_utils and calls model = load_model(). Use the preconfigured local model first; the public repository contains the notebook, but the supporting modules and model file must be verified inside the image.
4. Run the notebook's model-path and build checks. It checks MODEL_PATH, USE_BASE_MODEL, GGUF file magic, llama_cpp_python, and a zDNN shared library. Record the actual model identifier or path, version, quantization if available, image digest, and backend evidence.
5. Run a short local generation using the notebook's supported functions and confirm it does not call an external inference service. The notebook uses BM25 retrieval for policy chunks; the speaker's description of a vector database is broader than the checked-in implementation.
6. Stop the heavy model or restart its kernel before a large Spark job. The 8 GB session allocation is limited. Persist final notebooks, prompts, and outputs in shared and back them up with the code repository.

```
import sys, platform, importlib.metadata as md
print(sys.version)
print(platform.machine())
for package in ["jupyterlab", "llama_cpp_python", "torch", "numpy"]:
    try:
        print(package, md.version(package))
    except md.PackageNotFoundError:
        print(package, "not installed in this kernel")
```

VERIFY: no exact Granite model release, quantization, context limit, or model download URL was stated clearly enough in the video to standardize here. Do not assume an arbitrary Hugging Face GGUF or an x86 wheel works on s390x. The packaged image is the supported first path. A backend library or NNPA CPU flag establishes availability; verify actual dispatch and measurements before claiming our workload uses it.

Performance correction [V 44:49 to 45:11; S9]: the screen shows a batch of 1,000 images, median batch time 3.63 ms, best batch time 3.50 ms, and about 275,504 images per second. The spoken phrase per image is inconsistent with that display. The notebook computes throughput as batch size divided by batch duration. These are presenter demonstration results, not OutOfRole performance or a guaranteed VM speed.

7 Obtain and inspect the CERT data

This section expands the existing OutOfRole data-access guide [L1]. The chosen source remains CMU SEI's synthetic CERT Insider Threat Test Dataset release r4.2 and its answer key [S13]. The local archive metadata and extracted files were checked. The release README warns that it is a dense-needle dataset with unrealistically frequent malicious activity, and some logons and device disconnects are intentionally missing.

Download	Published size	MD5 from CMU metadata
r4.2.tar.bz2	4,824,287,500 bytes	cf64caa378acb77cd0c608a5576d998c
answers.tar.bz2	1,254,678 bytes	c0c96eaa06915e417504f932756f58d8

Published download links and the README link from the original guide are retained in Appendix C. MD5 is used here for consistency with CMU's metadata; the commands below also check the published SHA-256 values. Checksums detect a mismatch; they do not change the data license.

Download directly on the VM

1. Use the mounted project folder ~/shared/outofrole so artifacts are visible in Jupyter. The earlier guide used ~/outofrole; either location can work, but keep one canonical root and update every path consistently. Existing laptop files do not automatically appear on the VM.
2. Check free disk before downloading. The complete compressed archive is about 4.82 GB in decimal units or 4.49 GiB; full extraction is roughly 16.3 GB in decimal units before Docker images, models, Spark spill, and output. The original guide's 20 GB estimate is tight for a full extraction. Prefer selective extraction for the first version.
3. Download the two archives using the exact CMU links. curl -C - resumes an interrupted transfer. If a server refuses resume, preserve the partial file under another name and restart a fresh download after checking disk space.
4. Run the SHA-256 checks before extraction. Both files must report OK. A mismatch means stop, compare against the official metadata, and redownload the affected file.

5. List the archive contents if needed, then extract only logon, device, file, LDAP, readme, and license for the minimum project. Extract the small answer archive separately. Do not mix releases.
6. Read the release README and license, then inspect the real header and a small number of rows. Keep raw data and answer keys out of the Git repository.

```
mkdir -p ~/shared/outofrole/data/raw
cd ~/shared/outofrole/data/raw
df -h .
curl -fL -C - -o r4.2.tar.bz2 \
  https://ndownloader.figshare.com/files/24856766
curl -fL -C - -o answers.tar.bz2 \
  https://ndownloader.figshare.com/files/24857828
sha256sum r4.2.tar.bz2 answers.tar.bz2
```

Compare the full hashes below, or save them in a checksums file and run sha256sum -c against it. On a Mac use shasum -a 256 for SHA-256 or md5 -q for the original MD5 checks; Linux uses sha256sum and md5sum.

```
586446b00e83ce5cdabf9dc889b0b6e7a9f44556cee3c03d39445d790122556c
  r4.2.tar.bz2
ed21bf921def01e97326b314f2d1fa77651a945bd31bf513095f8ac7658c9ddd
  answers.tar.bz2
```

The display above puts each filename on its own line for readability. A sha256sum -c file requires the hash and filename on the same line separated by two spaces. Use the following executable Python check to avoid a formatting mistake.

```
python3 - <<'PY'
from pathlib import Path
import hashlib
expected = {
    "r4.2.tar.bz2":
      "586446b00e83ce5cdabf9dc889b0b6e7a9f44556cee3c03d39445d790122556c",
    "answers.tar.bz2":
      "ed21bf921def01e97326b314f2d1fa77651a945bd31bf513095f8ac7658c9ddd",
}
for name, wanted in expected.items():
    h = hashlib.sha256()
    with Path(name).open("rb") as f:
        for block in iter(lambda: f.read(8 * 1024 * 1024), b""):
            h.update(block)
    if h.hexdigest() != wanted:
        raise SystemExit(f"CHECKSUM MISMATCH: {name}")
    print("OK", name)
PY

mkdir -p ../r4.2 ../answers
tar -xjf r4.2.tar.bz2 -C ../r4.2 --wildcards \
  '*/logon.csv' '*/device.csv' '*/file.csv' '*/LDAP/*' \
  '*/readme.txt' '*/license.txt'
tar -xjf answers.tar.bz2 -C ../answers
cd ~/shared/outofrole
head -n 3 data/r4.2/r4.2/logon.csv
head -n 3 data/answers/answers/insiders.csv
```

Expected roots match the original guide: data/r4.2/r4.2 and data/answers/answers. Listing an entire 4.82 GB bzip2 archive or selectively extracting it still requires substantial decompression work; selective extraction mainly saves disk. Download time depends on actual bandwidth. The prior 10 to 15 minute estimate is not a service guarantee.

Files to use and what they mean

Source	Verified size or count	Meaning for the project
logon.csv	58.5 MB; 854,859 rows	id, date, user, pc, activity; logon and logoff behavior

Source	Verified size or count	Meaning for the project
device.csv	29.0 MB; 405,380 rows	id, date, user, pc, activity; removable-device connect and disconnect
file.csv	193.1 MB; 445,581 rows	id, date, user, pc, filename, content; every row is a copy to removable media
LDAP	18 monthly snapshots	December 2009 through May 2011; employee_name, user_id, email, role, business_unit, functional_unit, department, team, supervisor
psychometric.csv	1,000 employees; 44.7 KB	Big Five personality variables; exclude from our first version
email.csv	1.36 GB	Stretch; id, date, user, pc, to, cc, bcc, from, size, attachments, content
http.csv	14.54 GB	Stretch; id, date, user, pc, url, content; largest source

The source README calls the email count attachment_count, while the actual r4.2 header is attachments; code must use the real header. Dataset dates span roughly January 2010 to May 2011. Preserve source-local time semantics because a geographical timezone is not established. Do not visit the synthetic URLs: the README warns randomly generated domains can resolve to malicious sites.

Answer key and evaluation scope

Verified locally: insiders.csv has 70 rows for dataset 4.2, with 30 scenario 1, 30 scenario 2, and 10 scenario 3 insiders. Its columns are dataset, scenario, details, user, start, end. The scenario folders contain variable-length event records; csv.reader can parse them as row lists, but a fixed rectangular DataFrame is inappropriate without normalization.

Malicious event source	Labeled rows
logon	198
device	2785
file	10
email	470
http	3860
All five sources	7323
Minimum project sources only	2993

The original abstract's 7,323 labeled actions is verified across all sources. The minimum project observes only 2,993 of them. Do not report recall over all 7,323 as if every source were ingested, and do not imply all 70 insiders appear in a particular held-out period. Recalculate eligible positives after the split and coverage filters. The earlier 32.8 million total-events figure was not recounted across the two largest logs and remains VERIFY.

Scenario summaries retained from the original guide: after-hours device use with uploads to wikileaks.org; job hunting followed by heavy removable-media theft; and a disgruntled administrator using a keylogger. Those mechanisms do not all remain observable when email and HTTP are excluded.

License and repository housekeeping

The bundled ExactData license restricts redistribution and favors describing general qualities of the data. Follow its actual terms. Each teammate or VM should obtain data from CMU rather than publish copies. Use synthetic demonstration examples or aggregates where practical. When reproducing a minimal necessary source excerpt, retain the copyright notice specified by the license: Copyright 2011 ExactData, LLC, All Rights Reserved. These are source-license constraints, not IBM event rules.

```
# Add these patterns to the repository .gitignore.
data/
.venv/
.env
*.pem
*.ppk
*.key
*.gguf
__pycache__/
```

Before every push, inspect git status and staged files. Credit CMU SEI CERT Insider Threat Test Dataset r4.2, its dataset page, DOI 10.1184/R1/12841247, and applicable notices in the README and presentation. A repository license for our code does not relicense the dataset.

8 Configure PySpark for OutOfRole

TEAM SUGGESTION: PySpark and ALS come from our existing abstract [L2]; they were not demonstrated or required in this video. The official image package list checked for this edition does not list pyspark. Use an explicit environment instead of assuming Jupyter already contains it.

The simplest reproducible first path separates two runtimes on the same VM: Spark runs in a host Python environment; Jupyter and Granite run inside the IBM container. They exchange files through ~/shared/outofrole. The container cannot automatically import packages installed in the host environment. This separation also makes resource use easier to control.

1. On the Ubuntu VM, install Java 17, Python venv, and the distribution NumPy package. Apache Spark 3.5.6 documents Python 3.8 or newer and Java 8, 11, or 17 [S14]. This guide chooses Java 17 and pins PySpark 3.5.6; using latest silently changes the compatibility contract.
2. Create the virtual environment with access to system packages so Ubuntu's s390x NumPy can be reused. Install PySpark in that environment. If pip must compile an unsupported native dependency, stop and check platform packages rather than downloading an amd64 wheel.
3. Run the small local Spark check below from the host shell. It deliberately reserves two local worker threads and a 2 GB JVM heap as a starting point for the 8 GB VM, not a guarantee of sufficient memory for the final job.
4. Save the successful Java, Python, Spark, NumPy, and OS versions. Write aggregates and metrics under the shared artifacts directory so the notebook can read them.
5. For a later notebook-only Spark setup, install a compatible JVM and PySpark inside the same container or build a derived image. Re-run the Spark check there. Registering a host kernel alone does not make that kernel executable inside the container.

```
sudo apt-get update
sudo apt-get install -y openjdk-17-jre-headless \
  python3-venv python3-numpy
mkdir -p ~/shared/outofrole/artifacts
cd ~/shared/outofrole
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
python -m pip install 'pyspark==3.5.6'
java -version
python -m pip show pyspark numpy
export PYSPARK_PYTHON="$PWD/.venv/bin/python"
export PYSPARK_DRIVER_PYTHON="$PYSPARK_PYTHON"

python - <<'PY'
from pyspark.sql import SparkSession
spark = (SparkSession.builder.appName("OutOfRoleSmoke")
    .master("local[2]")
    .config("spark.driver.memory", "2g")
    .config("spark.sql.shuffle.partitions", "4")
    .getOrCreate())
assert spark.range(1000).count() == 1000
print("Spark", spark.version, "smoke check passed")
spark.stop()
PY
```

If Java is not found, inspect the installed path. JAVA_HOME must name the JVM root, not its bin/java executable. On Ubuntu s390x, readlink -f /usr/bin/java identifies the selected Java installation. Set JAVA_HOME to that installation's root if the launcher cannot infer it. Do not copy an x86 path ending in amd64. In each new SSH session, run source ~/shared/outofrole/.venv/bin/activate again and reapply the two PYSPARK interpreter exports from the project folder.

VERIFY: Java and s390x native-library behavior must be checked on the actual VM. A warning that a native Hadoop or BLAS library is unavailable does not by itself prove the job failed; confirm the count check and then test ALS. Spark MLlib ALS is a JVM algorithm and is not automatically accelerated by NNPA. Do not claim an NNPA speedup merely because lscpu shows the instruction facility.

9 Implement OutOfRole during the event

This is a proposed implementation runbook, extending the original abstract rather than reporting completed code or measured results. The abstract compares the idea with a recommender: activities inconsistent with learned relationships become review candidates. Our implemented claim should be unusual workstation and removable-media behavior in synthetic enterprise logs. Patient records, bank-account access, permissions, and true business authorization are not present in CERT r4.2.

Stage 1 Define the demo and preserve provenance

1. Define the analyst question: Which user activities are most unusual relative to the user's own history and current organizational peers, and what evidence should an analyst review?
2. Create notebooks, src, artifacts, and tests folders during the coding window. Keep raw data immutable. Record dataset release, checksum, environment versions, source license, code commit, and chosen time split.
3. Define success using held-out precision, recall, alert burden, and scoring coverage. Set an alert budget, such as the top 20 user-days, as a team design choice and report that choice with results.
4. Keep the proposed least-privilege feature as future work unless a real permissions dataset is supplied. A missing recommendation does not prove a permission is unnecessary, and this prototype must not remove access automatically.

Stage 2 Normalize activity and attach role context

1. Load logon, device, and file CSVs with explicit schemas. Parse dates using MM/dd/yyyy HH:mm:ss, retain the source timestamp string, and count invalid dates before dropping or quarantining them. Do not silently interpret them as Chicago time.
2. Create a canonical event record containing source, source_event_id, event_time, user, pc, activity or resource, and source provenance. The release README says IDs may repeat across CSV files, so use the pair source and source_event_id as the key.
3. Map successful logons to a resource name such as pc:PC123. Map file copies to a stable resource such as file:normalized_path or a documented path category. Keep PC and file namespaces distinct. Fit normalization rules on training data.
4. Aggregate device connect counts and file-copy counts per user and day, after-hours behavior relative to that user's learned schedule, new workstations, and deviations from role or department peers. Missing disconnects do not establish malicious behavior.
5. Join LDAP with user equal to user_id using the most recent snapshot available at or before the event. Keep role changes and missing metadata visible; do not apply a future month's role or termination information to earlier events.
6. Persist cleaned data or features as Parquet and retain the ingestion counts. In notebooks, inspect small samples or bounded aggregates rather than calling toPandas on the full logs.

```
# Illustrative ingestion cell or host-script fragment.
from pyspark.sql import functions as F
schema = "id STRING, date STRING, user STRING, pc STRING, activity STRING"
logon = (spark.read.option("header", True).schema(schema)
    .csv("data/r4.2/r4.2/logon.csv")
    .withColumn("event_time",
                F.to_timestamp("date", "MM/dd/yyyy HH:mm:ss")))
bad_dates = logon.filter(F.col("event_time").isNull()).count()
print("Unparsed dates", bad_dates)
logons = (logon.filter(F.col("activity") == "Logon")
    .withColumn("source", F.lit("logon"))
    .withColumn("resource", F.concat(F.lit("pc:"), F.col("pc"))))
```

Declare the Spark session timezone used for reproducible timestamp storage, and describe it as an assumption when the dataset timezone is unknown. The fragment above presumes an existing SparkSession from Section 8; it is not a complete pipeline.

Stage 3 Split time and build a baseline

1. Choose chronological cutoffs before model selection. An illustrative plan is training before June 1 2010, validation during June 2010, and testing from July 1 2010 onward. Check that each period has adequate observations. These are team suggestions, not organizer dates.
2. Fit encoders, resource categories, user schedules, peer statistics, scalers, and model parameters using training data only. Choose thresholds and weights on validation data; evaluate the held-out test once the design is fixed.
3. Build an interpretable baseline that ranks a new workstation, an unusual time for that user, high file-copy volume, and unusual device connections. Explain each component with counts and comparison windows.
4. Keep termination and role changes time-correct. Do not use an insider's future incident interval, scenario, or identity to train an unsupervised detector. Persist both the baseline and its alert budget so ALS has a fair comparator.

Stage 4 Train an implicit feedback ALS model

ALS means alternating least squares. It learns latent user and resource factors from observed interactions. In implicit feedback, repeated activity increases confidence that an interaction is normal; it does not provide a verified authorization label. Low predicted affinity can suggest unusual behavior, but its score is not a calibrated probability of an attack [S15].

1. For the training interval, group user and resource interactions into a nonnegative count or a documented transform such as log1p(count). Cap extreme frequencies if validation shows they dominate. Persist the exact definition.
2. Fit and save separate training-only StringIndexer models for users and resources. Convert their indices to integer user_idx and resource_idx columns; Spark ALS requires IDs within the signed integer range. Preserve reverse mappings for explanations.
3. Start with a small rank such as 8, five iterations, and a limited number of user and item blocks. Tune only if the small model runs and validation supports a change. The parameter values below are proposed starting values, not measured optimums.
4. Keep coldStartStrategy as nan during scoring. New users or resources lack trained factors; mark them explicitly and route them to peer or novelty baselines. Dropping them silently can hide exactly the unusual events we want to inspect.
5. Rank low-affinity known interactions, then combine that evidence with baseline signals. Calibrate the ranking or threshold on validation. Do not label every previously unseen file or workstation as an intrusion.

```
from pyspark.ml.recommendation import ALS
als = ALS(
    userCol="user_idx", itemCol="resource_idx",
    ratingCol="strength", implicitPrefs=True,
    rank=8, maxIter=5, regParam=0.1, alpha=10.0,
    numUserBlocks=2, numItemBlocks=2,
    coldStartStrategy="nan", seed=42)
# training_interactions must be encoded from training data only.
model = als.fit(training_interactions)
scored = model.transform(encoded_heldout_events)
cold = scored.filter(F.isnan("prediction") | F.col("prediction").isNull())
known = scored.filter(~F.isnan("prediction") & F.col("prediction").isNotNull())
```

Persist the model, mappings, configuration, and training cutoff. Avoid using random row splits for this temporal detection problem. Keep separate counts for input events, scored known events, cold-start events, rejected rows, and alerts so coverage can be audited.

Stage 5 Explain alerts locally with Granite

1. Adapt the loan notebook's local-model pattern to security review. Send only an alert's structured evidence: source event IDs, role and department as of the event, time-window counts, baseline values, model affinity or rank, and reason codes.
2. If policy retrieval is useful, write a small fictional demonstration policy and label it as team-authored. Chunk it and retrieve relevant passages. Do not present that policy as an actual hospital, bank, IBM policy, or ground truth from CERT.
3. Ask Granite to summarize observations, distinguish facts from possible explanations, identify missing context, and suggest a human review step. Do not request a verdict on guilt. Keep raw file content, emails, and personal names out of prompts unless demonstrably needed.
4. Validate each generated number and event reference against the structured alert. Flag unsupported claims and retain the deterministic explanation if generation fails. Record prompt version, model details, latency, and whether the model ran locally.

5. Test a routine role change, a new employee, an administrator, and high but legitimate removable-media use. Treat log text as untrusted data: instructions inside a filename, message, or policy excerpt must not override the explanation task.

```
Suggested prompt structure
You assist a human security analyst reviewing synthetic activity.
Use only the supplied evidence and cited demonstration policy.
Do not infer intent or invent permissions, facts, events, or numbers.
Return observations, why review may be useful, benign alternatives,
missing context, cited event IDs, and a suggested human review step.
If the evidence is insufficient, say so explicitly.

EVIDENCE_JSON: <validated alert fields>
POLICY_EXCERPTS: <retrieved passages with document IDs>
```

The placeholders above are inputs to our future code, not literal production prompts. A rule-based explanation is a valid fallback; mark whether a displayed explanation is generated, deterministic, or cached. The claim that data stays on the platform requires actual local inference and controlled outputs. It does not establish HIPAA, GLBA, confidential-computing, or encryption-in-use compliance.

Stage 6 Evaluate without leaking the answer key

1. Normalize each selected r4.2 answer-detail row using csv.reader. The first field is source, the next is the source event ID, followed by its date and user. Retain at least source, id, user, scenario, and event time; verify each record's length before indexing.
2. Deduplicate labels by the composite source and id key, then join to only the sources and dates actually scored. Retain unmatched label counts and investigate a source-name, ID, or date parsing mismatch.
3. For event-level metrics, compute TP, FP, and FN only on eligible test events. Precision is TP divided by TP plus FP; recall is TP divided by TP plus FN. If a denominator is zero, report undefined with its support count rather than inventing a value.
4. If the analyst view ranks user-days, aggregate labels to the same user-day level and compute metrics at that level. Report event recall and user-day recall separately. For user-level detection, state the number of eligible insiders in the held-out period.
5. Compare the baseline, ALS alone, and combined ranking at the same alert budget. Report precision at K, recall at K, alert count, false positives per benign user-day, and cold-start coverage. Use PR-AUC only if a consistent continuous score and full eligible labels are available.
6. Measure startup separately from warmed inference, then report median and p95 scoring or explanation latency with batch size, row count, CPU and RAM allocation, and whether it is a replay. Anomaly performance and explanation quality are separate evaluations.

The original abstract proposes replaying later months as a live stream. A time-ordered batch replay is sufficient for a first demo if clearly labeled simulated or replayed. Add Structured Streaming only if it works, preserves checkpoints, and fits the event window. Do not describe a replay as a live production feed.

Stage 7 Present an analyst workflow

1. Show a ranked list with time, pseudonymous user, source, risk rank, reason, and cold-start status. Open one alert to show its contributing events and baseline comparison.
2. Show one true positive and one false positive or legitimate unusual case selected from the evaluation set after scoring. Explain that cherry-picked examples do not replace the full metrics.
3. Run the pipeline or a bounded replay on LinuxONE while recording; show s390x and the actual environment without exposing secrets. Show that the model and explanations use our own OutOfRole data flow.

4. Display dataset limitations: synthetic activity, dense malicious examples, no real patient-record access, no complete permissions model, and reduced visibility when email and HTTP are omitted. Explain what a real deployment would additionally require.

10 Troubleshooting and recovery

Start with the smallest failing step. Record the exact error, command, timestamp, OS, image digest, and last successful check. Remove tokens, private keys, passwords, and sensitive outputs before posting in Discord. The video directs support to IBM SMEs there [V 54:05 to 55:06].

Account VM and SSH problems

Symptom	Checks and next action
No email or invalid credentials	Check spam and activation, email spelling, and the SSF2026 event code. The presenter expects normal activation in seconds to about 10 minutes. Use linux1@us.ibm.com or the SME channel for a stuck account.
No AI Lab flavor	Likely missing event entitlement. Ask support to correct or reset registration as described in the session. Do not keep provisioning small trial VMs.
VM takes a long time	Refresh the dashboard after the presenter's 15 to 20 minute busy-period allowance. Check ACTIVE or SHUTOFF. Avoid repeated Create clicks.
Permission denied publickey	Use linux1, the current VM IP, and the private key matching the selected portal key. Check chmod 600 on Mac or Linux and the SSH -i path. Portal passwords do not replace the key.
Host identification changed	If the VM was genuinely rebuilt or the IP reused, verify the new fingerprint first, then remove only that stale host with ssh-keygen -R VM_IP. Do not turn off checking.
SSH timeout	Check ACTIVE, current IP, laptop network or VPN, and TCP 22. Ping can be filtered and is not a complete test. IBM's lab FAQ suggests Stop then Start for a hung ACTIVE VM; coordinate first because this interrupts teammates.

Docker and Jupyter problems

1. If Docker reports socket permission denied, reconnect SSH after usermod. Check id and systemctl status docker. Do not chmod the Docker socket to be world writable.
2. If notebook is not running, inspect docker ps -a and docker logs --tail 100 notebook. Correct an invalid command or port before recreating it. Confirm work is on the shared host mount; files stored only in the container may be lost on replacement.
3. For a shared-folder permission error, compare id on the host with docker exec notebook id. If the image user is UID 1000 as IBM's FAQ expects, set ownership on the specific shared project folder appropriately. If the IDs differ, use the actual numeric IDs and preserve needed team access. Do not recursively change all of /home/linux1.
4. If the browser cannot connect, verify the SSH tunnel is still open, the chosen laptop port matches the URL, and Jupyter listens on VM loopback port 38888. The private tunnel route does not require a new public firewall rule.

5. If authentication fails, use the actual token passed at startup. The printed example Your_Token1 is not our token. docker logs can disclose tokens, so redact before sharing. A changed shell variable does not update a running container's token.
6. If a container name already exists, inspect it first. docker start notebook can resume it. Replacing it requires stopping and removing that specific container after confirming persistence. Avoid docker rm -f notebook while someone is executing a cell.
7. For image architecture or illegal-instruction errors, verify s390x, the selected image, and the Datathon OS. For seccomp errors, preserve the error and consult an SME before using the README's optional seccomp=unconfined flag.

```
docker ps -a
docker logs --tail 100 notebook
docker exec notebook id
docker inspect notebook --format '{{json .Mounts}}'
ss -ltn | grep 38888
docker stats --no-stream
free -h
df -h "$HOME"
docker system df
```

Targeted shared folder permission fix

TEAM SUGGESTION: if the host and image user IDs differ, a Linux access control list can grant the container user access without changing the owner or making the folder public. Run this on the Ubuntu VM, before creating new project files. Set IMAGE as in Section 5. It changes only ~/shared and its defaults for future children; preexisting nested folders may need the same targeted fix separately. If both user IDs are equal, this step is usually unnecessary.

```
sudo apt-get install -y acl
OUTOFROLE_HOST_UID=$(id -u)
OUTOFROLE_CONTAINER_UID=$(docker run --rm --entrypoint id "$IMAGE" -u)
setfacl -m "u:$OUTOFROLE_CONTAINER_UID:rwx" "$HOME/shared"
setfacl -d -m \
  "u:$OUTOFROLE_HOST_UID:rwx,u:$OUTOFROLE_CONTAINER_UID:rwx" \
  "$HOME/shared"
getfacl "$HOME/shared"
```

Data Spark Granite and disk problems

1. For a missing CSV, check the doubled r4.2 extraction root and the host versus container path. The Jupyter web upload limit of 200 MB is a statement in the IBM lab FAQ, not a universal Jupyter limit. Download our 4.82 GB archive directly to the VM or use scp for small files.
2. For JAVA_GATEWAY_EXITED, check java -version, the JVM architecture, JAVA_HOME, and the Python interpreter. Run the 1,000-row smoke check before ALS. Do not run Spark in a container kernel that lacks its own JVM.
3. For NaN ALS scores, count new users and resources and check saved training encoders. Route cold starts to the fallback instead of dropping them. For integer-ID errors, inspect casts and invalid-index handling.
4. For Spark memory pressure, use fewer columns, smaller partitions, lower ALS rank or blocks, and bounded datasets. Avoid caching every stage or collecting all rows to the notebook. Stop idle Granite and notebook kernels during a large fit.
5. For Granite missing modules or model files, inspect the bundled loan_demo directory and MODEL_PATH from the provided notebook. For a native-library or endian error, use the supported IBM image and consult its maintainers; installing generic x86 binaries is not a fix.

6. For disk pressure, inspect archive, raw extraction, model, image, and Spark spill sizes before deleting anything. Remove only identified unused artifacts after backing up important files. Do not run the old FAQ's blanket image removal or aggressive volume pruning command.
7. After a kernel restart or VM reboot, reopen the saved notebook, restore the correct environment, restart the intended service, and rerun only the required setup cells. Recover source and configuration from Git; the cloud has no backup guarantee.

Useful small-file transfer from a laptop: scp -i YOUR_KEY.pem notebook.ipynb linux1@VM_IP:~/shared/. Replace the key path and IP. Avoid uploading raw data or credentials into a public help channel. The source readme's memory-cache-clearing shell example is not reproduced because dropping OS caches does not solve a job's working-set memory requirement.

11 Judging evidence and final submission

PRESENTER [V 52:07 to 53:19]: the slide shows four categories with a maximum of 10 each. Alex calls Technical Design the most heavily weighted area, but no numeric weights are given. The official event FAQ lists related factors without weights. Do not invent percentages or a tie-break formula.

Video category	Presenter recommendations from the slide	Our proposed evidence
Technical Design max 10	Show live LinuxONE work; use at least two AI open-source frameworks; show Docker or Podman deployment; highlight security or Telum acceleration	Spark MLlib detector plus local Granite runtime; actual image and platform proof; local data flow and measured latency
Novelty and Originality max 10	Write solution code during the event; adapt reference labs; combine predictive models with Granite reasoning	Original role-aware recommender workflow, baseline comparison, source history, prompt and explanation validation
Pitch and Delivery max 10	Observe the published video limit; state problem, persona, value; show interactive LinuxONE demo; credit team contributions	Short analyst scenario, real execution, metrics and limitations, contribution slide, accessible links
Community Value max 10	Identify a quantified beneficiary group; tie the application to one theme; frame AI for Good; use ethical AI and transparent reasoning	Security analysts and privacy officers; measured alert burden and coverage; human review and data minimization

The slide calls these actionable recommendations. Treat the at-least-two-frameworks advice as PRESENTER, not a verified eligibility rule. Spark MLlib and llama.cpp or llama_cpp_python provide a plausible two-framework design if actually implemented. Docker and Jupyter alone do not establish two AI frameworks. Security features or NNPA acceleration must be evidenced, not asserted from platform marketing.

Prepare the final assets

1. Open the live Hack2skill final-project submission section and read every required field, deadline, video length, file-size limit, allowed link type, and access rule. The public FAQ currently calls it Abstract submission section even for final submission; do not overwrite the accepted abstract without confirming the correct module.
2. Prepare the working prototype or demo, repository link, short description, and pitch deck explicitly listed in the official event FAQ [S2]. The workshop also expects a demo video, screenshots of LinuxONE

execution, and compliance with the published video time limit. Exact video duration and upload-size limits were not verified.

3. Complete the repository README with the problem, audience, theme, architecture, why LinuxONE, data source and license, exact setup commands, versions and image digest, how to run, evaluation split and support counts, metrics, limitations, and team contributions.
4. Include source code and necessary small configuration or dependency files. Keep data, answer keys, tokens, SSH keys, downloaded model weights, and private outputs out of the repository. Save only licensed and appropriately anonymized demonstration assets.
5. Show the real system working on LinuxONE and the exact proposed workflow: inspect an unusual event, show ranked evidence, and explain the alert locally. Label precomputed or cached results and replayed data. If a feature failed, state its status rather than presenting a mockup as execution.
6. Test repository, demo, recording, and deck links in a signed-out or appropriate reviewer session. Ensure the access setting matches the portal's instructions; public access is not assumed when the rules do not specify it.
7. Enter the final form, attach or link all required assets, review the preview, and submit before the confirmed deadline. A saved draft is not proof of submission. Save the confirmation screen, receipt, timestamp, and submitted commit or release.
8. Have a second teammate verify the receipt and links. Keep a backup of the submitted version. Follow organizer instructions for any later corrections; do not assume you can edit after the cutoff.

Suggested recording order

TEAM SUGGESTION: allocate the confirmed video time limit proportionally: roughly one sixth for the problem and users, one sixth for data and architecture, one third for the actual LinuxONE workflow, one sixth for evaluation and limitations, and one sixth for impact and contributions. Adjust after reading the actual limit; this does not imply a specific required number of minutes.

Use our original audience framing carefully [L2]: security analysts, privacy and compliance teams, identity administrators, and people whose records deserve protection. CERT demonstrates an enterprise insider-threat analogue, not an actual healthcare or bank audit. Local processing may reduce data movement, but we have not demonstrated regulated compliance or a production IBM Z records integration.

Appendix A Cleaned topic transcript with timestamps

Primary source [V]: LinuxONE Setup and Tech Prep Session, Shooting Stars Foundation, October 16 2026. YouTube exposed no manual or automatic captions through yt-dlp. A complete 662-fragment audio stream and matching video stream were recovered; Whisper small.en generated timestamped text and SRT locally. All recovered speech was reviewed as text, and key slides and commands were checked against the video. This is an edited, paraphrased topic transcript, not a verbatim quotation. Repetition and screen-sharing pauses are condensed; meaningful explanations, warnings, numbers, corrections, and the closing question exchange are retained.

Timestamp convention: mm:ss relative to recovered media, which lasts 55:06. YouTube metadata says 55:28. We could not verify the extra 22 seconds or establish a reliable offset; use timestamps as approximate navigation into the linked video and check nearby content. Speech-recognition mistakes such as Portman, Talam, Onyx, and Livingstone were normalized to Podman, Telum, ONNX, and LinuxONE using the slides and official sources. Unclear nontechnical fragments were not reconstructed as instructions.

LinuxONE and IBM Z scope

00:00 to 00:48 Alex Osadchyy explains that the two platforms share architecture. IBM Z can run both proprietary z/OS applications and open Linux workloads, while LinuxONE focuses on Linux. The session will cover architecture, access, reference machine-learning labs, themes, judging, and submission advice. The recording begins mid-explanation; an earlier introduction is not recoverable from the downloaded media.

Platform capabilities and event access

00:51 to 03:41 Alex describes LinuxONE as an enterprise platform for cloud-native applications, AI, and data-intensive workloads. Linux distributions and open frameworks include Ubuntu, Red Hat, Fedora, containers, Kubernetes, PyTorch, TensorFlow, and XGBoost. He highlights pervasive encryption, reliability, and hardware offload for encryption, compression, and AI. Telum includes an on-chip AI accelerator. Participants receive free access through LinuxONE Community Cloud and can build on infrastructure used in banking, insurance, government, travel, retail, and other enterprise settings. Verification note: these are platform capabilities. Encryption in memory, confidentiality, accelerator dispatch, and compliance depend on the actual configuration and application; free trial access does not establish all of them.

Lab architecture and access methods

03:41 to 05:31 Alex describes LinuxONE 5 machines in the Community Cloud with Telum AI acceleration. Participants provision a Linux VM on that hardware, choose Red Hat or Ubuntu for this event, and bring up Docker or Podman. The prepared container contains more than a hundred packages, including Python, Jupyter, PyTorch, scikit-learn, TensorFlow, Keras, and llama.cpp for generative AI. The cloud portal and Jupyter use a browser; SSH or a terminal provides command-line access. Step-by-step documentation accompanies the environment.

Six steps to a submission

05:34 to 07:28 Alex gives the participant workflow: register for Community Cloud; deploy the Jupyter stack; run the four reference labs; build an original solution; verify the criteria; then prepare and submit the presentation. The examples cover predictive and generative AI. He advises checking criteria early, making AI central to the solution, running it on LinuxONE, and demonstrating that execution. Capture screenshots of work running on LinuxONE as submission evidence.

Bruce introduces the cloud allocation

07:31 to 09:42 Bruce Gilkes takes over. IBM partners with Marist University in Poughkeepsie, New York, to host the LinuxONE Community Cloud, OpenShift cloud, and an open-source software cloud. This workshop uses the first of those. He describes a real z17 or LinuxONE 5 system and an event trial of 30 days. The event VM allocation is one server with three virtual CPUs, eight gigabytes of RAM, and fifty gigabytes of disk. For this event choose Red Hat or Ubuntu, and use sudo cautiously because administrative commands can break the VM. The ecosystem of s390x open-source packages continues to grow. The cloud is broadly available internationally; separate event eligibility still applies.

Reference links and demonstration plan

09:42 to 10:32 Bruce recommends keeping the resource links available during the event. His slide lists the IBM Datathon FAQ, Community Cloud landing page, VM registration page, dashboard, AI and ML lab repository, virtual-server deployment guide, and Windows PuTTY guide. All readable URLs are retained in Appendix C. His prerecorded demo shows registration, dashboard access, server creation, SSH, and Linux inspection commands.

Registration form and project description

10:32 to 12:48 Bruce starts at the IBM Community landing page and selects Try Linux Virtual Machines. He enters an email and password and asks participants to remember them for the dashboard. The form also requests first and last name, organization or university, country or region, and a description of the project. The description is required and has a 75-character minimum. He asks for a few meaningful sentences because IBM reads the descriptions to understand usage and improve cloud content and capabilities.

Event code and a missing entitlement

12:48 to 14:10 Bruce emphasizes entering SSF2026 for this Datathon. The code speeds registration and entitles the account to the CPU and memory needed for the AI labs. A general trial outside an event may leave the code blank, but this event should not. If the registration was completed without it, contact LinuxONE support at linux1@us.ibm.com or a regional SME or Discord mentor. He describes having the registration deleted so the participant can register again with the correct code. Read the terms before requesting the trial. Verification note: the support address is normalized from the slide and IBM documentation; the audio spelling is ambiguous.

Activation email and dashboard credentials

14:13 to 15:31 Bruce waits for email, explaining that delivery may take seconds or five to ten minutes depending on load. Of the registration emails, the activation email is essential. Click its activation link. Without activation the registration remains incomplete, the credentials will not work, and a server cannot be deployed. After the activation confirmation, sign in using the credentials just created. The dashboard includes a View Guide link with deployment reminders.

Initial OS selection in the prerecorded demo

15:31 to 16:19 Bruce opens Manage Instances or Virtual Servers, selects Create, names the instance, and selects Ubuntu. The prerecorded narration mentions Ubuntu 24.04 and RHEL 9.6, explaining that supported Linux distributions are available but the event labs should use the latest Ubuntu or Red Hat image. This older RHEL selection is immediately corrected live.

Live correction to the Datathon images

16:19 to 16:47 Bruce pauses the recording for a just-in-time update: choose the images with the Datathon suffix. He identifies Ubuntu 24.04 Datathon and RHEL 9.8 Datathon. These are the correct event images. This correction supersedes the older RHEL 9.6 wording in the prerecorded demonstration.

AI Lab flavor and SSH key pair

16:48 to 18:33 Bruce selects Ubuntu and the AI Lab flavor. An account without the event code sees only the smaller Linux1 flavor, which does not provide the resources needed for these labs. Contact support to correct the registration if AI Lab is absent. For SSH, participants may import an existing public key or create a key pair in the portal. He demonstrates both: name and upload the public key, or generate a new pair and save the prompted private key file on the laptop. The selected public key is installed in the Linux server; the matching private key is used to log in.

Provisioning wait status and VM details

18:33 to 20:08 Bruce creates the instance. With potentially thousands of simultaneous requests, creation may take fifteen to twenty minutes. Take a break, refresh, and look for ACTIVE. A created instance may appear SHUTOFF and can be started from the controls. The dashboard also allows stopping and deleting an instance and shows the selected key and login user linux1. View Details exposes processor, RAM, disk, OS, and related information. Verification note: an unclear audio fragment during the transition is not treated as an instruction.

SSH on Mac Linux Windows and PuTTY

20:08 to 21:13 Bruce connects with ssh -i followed by the private key path and linux1 at the VM address. Mac, Linux, and Windows PowerShell use this same OpenSSH pattern and the same .pem private key. For Windows without that route, a separate guide explains PuTTY and converting the .pem into a .ppk. The guide is linked in Appendix C. The demonstration address belongs to Bruce's instance; participants must use their own dashboard IP.

A reused IP and known hosts warning

21:16 to 22:45 The demonstration encounters a host-key warning because that IP had previously belonged to another server. Bruce opens known_hosts and removes stale entries so the new host can be accepted. He explains that SSH protects against unexpectedly connecting to a different server. Team safety clarification: verify a legitimate rebuild or IP reuse and the new fingerprint before removing only the affected host entry. Do not delete the entire trust file or disable checking.

Root authority and rebuilding a broken server

22:46 to 23:22 Bruce demonstrates sudo -i to become root and warns that privileged commands can break the system. If a server is unusable, it can be deleted and recreated from the dashboard. He then leaves unnecessary root use aside: his next inspection commands do not need sudo. Team clarification: save important work before deletion; the official deployment guide says the cloud does not back up the VM.

CPU kernel virtualization and NNPA evidence

23:22 to 25:45 Bruce displays /proc/cpuinfo, the kernel or machine information with uname, and lscpu. The demonstration shows s390x or IBM S390, three assigned virtual CPUs, machine type 9175 identified as z17, and z/VM virtualization. He filters the CPU output for nnpa, explaining the Neural Network Processing Assist

facility and its availability to the VM. The reference labs use accelerator-enabled frameworks where supported. Team clarification: a CPU facility flag is evidence of access, not a measurement of acceleration in every algorithm.

One server per team

25:46 to 26:42 Bruce encourages participants to explore Linux commands and create their server. Before handing back to Alex, he asks each team to create one server, rather than one per user, so shared cloud resources are sufficient for everyone. Team members should share the working environment or designate an operator.

Alex introduces the notebook environment

26:42 to 28:43 After a screen-sharing transition, Alex explains the next phase: follow the GitHub instructions to start the Jupyter environment on the LinuxONE VM. Jupyter with Python is the main environment used in these examples. Run the prepared labs at your own pace to understand the approach, troubleshoot setup, and find ideas for an original project.

Fraud detection with historical transactions

28:43 to 30:37 The first highlighted example processes a synthetic IBM Research credit-card dataset described as about twenty-four million transactions, also available on Kaggle. The lab ingests and preprocesses data, then trains a two-layer LSTM. It learns from a sequence of seven transactions, including the history preceding a fraud event, instead of treating only the final transaction in isolation. The model runs on LinuxONE and can use supported Telum acceleration. No exact Kaggle dataset URL is given in the speech; none is reconstructed here.

Loan approval with predictive and generative AI

30:37 to 34:01 Alex highlights a new hybrid AI example. Structured numeric data, such as income and debt ratios, feeds a predictive model for risk scoring. Unstructured application documents are processed with IBM Granite. Lending-policy text is chunked and retrieved for context through RAG, or retrieval-augmented generation. The resulting evidence supports an approve, reject, or review recommendation with reasoning. Users can ask why a particular application was considered low risk or why a recommendation was made. He emphasizes local execution inside each team's isolated LinuxONE instance rather than sending inference to an external frontier-model service. Verification note: the checked-in loan notebook uses XGBoost and BM25 policy retrieval; its public source does not identify a particular Granite model file for this guide.

Churn profitability and traditional models

34:04 to 35:35 Alex describes a traditional machine-learning example using scikit-learn and PyTorch. Decision trees, random forest, and logistic regression are mentioned. Credit-card customer and transaction data is explored to estimate customer profitability and whether a customer may abandon the card. He uses the example to show that different business questions may need different kinds of models.

Handwritten digit classification

35:36 to 36:27 The image example uses the standard handwritten-digit setting with sixty thousand training images. It trains a TensorFlow or Keras convolutional model and predicts digits on LinuxONE. It is intended to demonstrate image classification and supported acceleration. Team clarification: the notebook also includes a later dense-network acceleration benchmark; do not assume every illustrated model or operation takes the same accelerated path.

Container deployment and token warning

36:27 to 38:42 Alex returns to the GitHub lab instructions, around Step 5, and copies commands into the LinuxONE terminal to pull and run the Jupyter container. docker ps verifies whether the container is running; the same README contains a FAQ for startup failures. Access Jupyter at the instance address and authenticate with its token. Because the VM is on a public IP, he specifically warns participants to choose their own token instead of retaining the default. The screen uses port 38888, a shared host directory, the s390x container image, and Docker. Current exact commands are cross-checked in Section 5 rather than reconstructed from speech alone.

Stepping through the churn notebook

38:42 to 41:46 Alex opens the notebook and executes cells in order. Explanations and code are interleaved, with displayed datasets, aggregate statistics, scatter plots, and correlation matrices. He illustrates an income-versus-age plot and discusses greater income in a middle-age range in this sample. He then examines correlation with the churn target, pointing out negative associations for age and activity level as candidate signals. The notebook proceeds to training and additional visuals. These are observations in a demonstration, not proof that the same features generalize to a new population or to insider-threat detection.

Trained model and churn probability

41:46 to 43:10 Alex encourages participants to run the documented cells themselves. He describes hybrid AI as combining predictive machine learning or neural networks with generative AI. The displayed logistic-regression example uses PyTorch, with improving training accuracy. Its plotted churn probability is higher for low activity and lower for high activity in this example. The notebook also demonstrates portable model export through PMML, visible on screen and described in the current lab README.

Digit model and ONNX portability

43:10 to 44:28 The digit notebook trains a two-layer convolutional network using TensorFlow or Keras, shows handwritten examples, and tracks improving accuracy. It exports a model to ONNX, an open interchange format. Alex describes portability as a way to move supported models from another development environment to LinuxONE when the original framework is unavailable there. Verification note: actual deployment still depends on supported ONNX operators, runtime packages, and the target architecture; export alone does not guarantee execution.

NNPA operations and the image benchmark

44:28 to 45:36 Alex names operations such as MatMul, Dense, ReLU, and Softmax as examples of acceleration capabilities and emphasizes a processor-adjacent accelerator and shared-cache advantages. The benchmark screen shows a batch of 1,000 images with median inference time 3.63 milliseconds, best time 3.50 milliseconds, and throughput 275,504 images per second. In speech, he calls roughly 3.6 milliseconds a per-image time while also citing about 275,000 images per second. The screen and notebook formula establish that the measured duration is for the full batch. The guide preserves the demonstration and corrects the unit rather than repeating the inconsistent wording as fact.

LSTM walkthrough and sequence throughput

45:36 to 47:03 Alex returns to fraud detection and explains long short-term memory with a history of small transactions followed by a larger example of 500 dollars. The notebook uses the historical context, trains on the transaction data, and demonstrates ONNX export. He reports around 10,000 sequences per second,

where each sequence contains seven transactions, on the limited-resource demonstration VM. This is a presenter result, not a measured OutOfRole result or a guarantee for every model.

Loan workflow and question answering revisited

47:05 to 49:51 The loan example combines structured values such as credit score, debt-to-income ratio, annual income, and requested loan amount with bank statements, pay stubs, tax forms, and lending policies. Policies are chunked for retrieval; unstructured features are extracted; XGBoost contributes structured risk scoring; and Granite combines the evidence. Outputs discuss financial stability, cash flow, income consistency, and policy considerations, leading to approve, reject, or additional review. A user can ask why the application was considered low risk and receive an explanation. Alex points out local token generation in the notebook but does not execute the full lab live because time is limited. He describes a vector database in narration; the source notebook checked for this guide uses BM25 retrieval.

Three Datathon themes

50:13 to 52:07 Alex asks teams to select one track. Real-Time AI for Critical Decisions emphasizes a flow of incoming data and immediate decision support, with payment fraud as an example. AI Secured Innovation Without Exposure emphasizes enterprise security and privacy with AI near the application core. Wildcard Beyond the Obvious allows other bold AI and data ideas, including food-waste prediction and a student-success coach. He says Adam will expand on the themes at kickoff. OutOfRole's existing abstract selects the second track.

Judging rubric and the submission playbook

52:07 to 53:19 The slide has four categories, each marked maximum 10: Technical Design; Novelty and Originality; Pitch and Delivery; Community Value. Alex emphasizes technical design and evidence that the project runs on LinuxONE, using security or Telum acceleration or both. The slide recommends at least two AI open-source frameworks, containerized deployment, and video screenshots of live work. For originality it advises solution code written during the hackathon, meaningful changes beyond reference notebooks, creative hybrid AI, and predictive models paired with Granite reasoning. For delivery it says to follow the published video time limit, explain the problem, target persona and value, show an interactive LinuxONE demo, and identify member contributions. For community value it asks for a specific quantified beneficiary population, a clear theme link, AI for Good, ethical AI, and transparent decision reasoning. Alex calls technical design the most heavily weighted criterion, but no percentages are shown or stated.

Final links and closing advice

53:19 to 54:05 Alex closes the technical enablement session, recaps the registration code, deployment guide, lab repository, dashboard, FAQ, and Community Cloud links, and says a recording will be available and may be replayed at kickoff. He encourages an original solution that works, has value, and gives participants a good experience. The final slide displays Your_Token1 as a sample token; this does not supersede the earlier warning to use a private token.

Audience questions are deferred to Discord

54:05 to 55:06 The moderator thanks Alex and Bruce and says that students have submitted questions in chat, asking if there is time for a few. Alex says he has another meeting and proposes using the Discord channels with IBM SMEs, including himself and Bruce. The moderator says the SME Discord support will be activated with the Datathon kickoff so participants can clarify questions, then closes and promises the recording shortly. No individual technical audience question is read aloud or answered in the recovered

recording. The chat itself was not retrieved, and its unseen questions or later answers are not invented in this guide.

Appendix B Verification and unresolved items

Sources were checked October 16 2026. Prefer a current official announcement or the actual submission form for event operations. Keep earlier claims traceable instead of silently rewriting them. The rows below identify where the guide deliberately adds a correction or qualification.

Item	Evidence and handling
Existing guide	Used the workspace data-access guide and project abstract as the available starting material. No separate Team Guide file was found; no claim is made to have merged an unseen document.
Transcript coverage	Recovered all 662 audio and video fragments in the successful streams. Whisper transcript reaches 55:06. Metadata says 55:28; extra 22 seconds unverified. No individual audience question or chat transcript recovered.
Image selection	Live correction requires Datathon-suffixed Ubuntu 24.04 or RHEL 9.8, superseding the prerecorded RHEL 9.6 selection.
Resource allocation	3 vCPU, 8 GB RAM, 50 GB disk, 30 days stated in video. Actual dashboard allocation and expiry not inspected.
Event code placement	Live demo and README use Event Code field SSF2026. Final slide also says project description; use the dedicated field when available.
Jupyter image	Current README uses /l1cc/ image path and /home/ibm-user/shared. Registry connection timed out from this computer; actual pull and image runtime untested.
Notebook security	The video warns against default tokens. Private loopback binding and SSH tunnel are team additions verified against OpenSSH and Jupyter documentation.
Model and acceleration	Public loan notebook shows local llama.cpp, MODEL_PATH checks, and zDNN checks. Exact Granite weights and actual acceleration on our VM are unverified.
RAG implementation	Video describes a vector database. Public loan notebook uses BM25. OutOfRole may use simple local retrieval if correctly identified.
Benchmark units	Video screen and notebook establish 3.63 ms for 1,000 images, not 3.63 ms per image. No OutOfRole performance was measured.
PySpark	Added from the team abstract, not a video requirement. Not in the checked package list. Versioned host setup is a proposed path, not a tested VM installation.
Data meaning and counts	Verified local core rows, 18 LDAP snapshots, 70 insiders and 7,323 malicious detail rows. Only 2,993 malicious rows are in minimum project sources. Full 32.8 million-event claim not recounted.
Submission deadline	Current event timeline gives Oct 18 16:30 UTC for project uploads. Top-level submissionEnd is abstract close; older FAQ dates conflict. Coding window and final form cutoff still require confirmation.

Item	Evidence and handling
Rules and format limits	Organizer official-rules page returned no substantive rule text to the tools. FAQ requirements and video recommendations remain separately labeled. Exact video duration, upload sizes, judging weights, final field layout, and Discord invitation URL are unverified.
Security and impact claims	Local execution is proposed and must be demonstrated. No automatic HIPAA or GLBA compliance, guaranteed encryption in use, NNPA speedup for ALS, production record access, or automatic permission removal is claimed.

For source conflicts, the team lead should record the organizer's resolution, source URL or channel, timestamp, and a screenshot in private event notes. Do not publish a source-provided credential or an uncertain support address. IBM infrastructure support is linux1@us.ibm.com; organizer support listed by the event site is stemprograms@sstarsfoundation.org.

Appendix C Source links and evidence register

V denotes the primary recording. S entries are official organizer, IBM-linked technical, or upstream documentation. L entries are the local starting material. Hyperlinks below are active in Word and PDF. Long URLs are attached to descriptive labels so they remain usable without overflowing the page.

V Primary workshop video

Shooting Stars Foundation; published October 16 2026. Recovered recording, Whisper transcript, and inspected slide frames are the primary evidence.

S1 Official organizer event hub

Event dates, themes, readiness and kickoff advertising, age 18 or older, official registration and rules links.

S2 Official Hack2skill event page

Public event page; dynamic visible content and FAQs recovered from its public event-details endpoint.

S2a Public event page data

Event page data source retrieved October 16 2026. Timeline, roster requirements, rules on pre-event work, submission assets, support, and eligibility. This is not a submission endpoint.

S3 Organizer official rules page

Linked by the organizer. Substantive rule text could not be extracted; do not infer missing rules.

S4 IBM LinuxONE Community Cloud landing page

Registration entry, s390x context, trial and support links. Current search-index text includes an Outlook and Hotmail registration alert.

S5 IBM linked virtual server deployment guide

Activation, key management, VM lifecycle, linux1 login, root and password-authentication restrictions, backups, and firewall behavior.

S5a IBM short deployment link shown in the video

Resolved to S5 when checked.

S5b VM registration link shown in the video

Cloud registration route shown on the resource slides; authenticated registration not performed.

S5c LinuxONE self service dashboard

Portal route shown in the video and IBM deployment guide. No account was accessed.

S6 Windows PuTTY setup guide

IBM-linked alternative to OpenSSH, shown on Bruce's resource slide.

S7 IBM LinuxONE Jupyter and AI lab repository

Current README checked directly, including SSF2026, Ubuntu 24.04, container path, mount, token and FAQ details.

S7a Documented image packages

Package snapshot retrieved for this edition. Versions can change with latest.

S8 Red Hat and Podman instructions

Current alternative branch; do not mix its SELinux and package-management instructions with Ubuntu.

S9 Reference loan notebook

Local model loading, path and backend checks, XGBoost and BM25 retrieval workflow.

S9a Reference digit notebook and benchmark

Verified batch size and throughput formula; video output establishes its displayed benchmark.

S10 Docker Engine on Ubuntu

s390x support, installation, convenience-script scope, service checks, and firewall caveats.

S11 Jupyter Server security

Token authentication and current IdentityProvider token option; retrieved directly.

S12 OpenSSH manual

Identity selection, local forwarding, and no-remote-command tunnel behavior.

S13 CMU SEI CERT dataset page

Dataset authority. Citation DOI 10.1184/R1/12841247. Local release README, answer README, and license were also inspected.

S13a CMU dataset metadata API

Exact file IDs, download URLs, byte sizes, and published MD5 values.

S13b r4.2 download

Original guide download retained and verified against CMU metadata.

S13c Answer key download

Original guide download retained and verified against CMU metadata.

S13d Published SHA256 file

Published SHA-256 values checked for both archives.

S13e SEI dataset README download

README link carried forward from the original guide; matching local SEI README inspected.

S14 Apache PySpark 3 5 6 installation

Version-specific Python and Java requirements; chosen host setup is a team suggestion.

S15 Apache Spark ALS documentation

Implicit feedback, integer IDs, latent factors, and cold-start behavior. Latest page can change; compare it with the installed Spark version.

S16 IBM Datathon FAQ shown in the video

URL preserved from slides. Tools returned a page shell without substantive text; source content unverified.

S17 Podman introduction

Upstream documentation for Podman's daemonless operation; Ubuntu remains the main path in this guide.

S18 Linux access control lists

Access and default ACL syntax for the optional targeted shared-folder fix.

L1 Data_Access_Guide.md, October 15 2026. Preserved dataset selection, download IDs, MD5 checksums, extraction paths, source schema, answer-key cautions, and license housekeeping; corrected size and evaluation-coverage qualifications.

L2 Abstract_Submission.md, October 2026. Preserved The Debugging Table, OutOfRole, AI Secured theme, recommender concept, local explanation idea, intended audience, implicit ALS, and later-period replay. The claims about patient records and permissions are retained as motivation and future deployment context, with the CERT data limitation stated explicitly.

Local verification evidence includes the actual r4.2 CSV headers and core row counts, 18 LDAP snapshots, 70 insider records, 70 detail files, and the per-source malicious-event count. Raw data and answer keys are not reproduced in this guide.